

Python Fullstack Developer

(Internship Report)

An internship report submitted in partial fulfilment of the requirements of IV B. Tech VI Semester of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

by

Yelamanchi Venugopal

23ME1A5466

Under Supervision of

Ms.G. Hema Sri

(Assistant professor)



Department of Artificial Intelligence & Data Science
RAMACHANDRA COLLEGE OF ENGINEERING

NH-16 Bypass Road, Vatluru(V), ELURU-534007, A.P.

Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK, KKD

Recognized by UGC 2(f) & 12(b)

Accredited by NAAC A+, NBA, ISO 9001 : 2015 Certified

2026-2027

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the Summer Internship Report entitled “ **Python Fullstack Developer**” by **Yalamanchi Venugopal** (23ME1A5466) is work done by him and submitted during academic year 2026-2027 Academic Year in partial fulfilment of the requirements of IV B.Tech VI Semester of Bachelor of Technology in Artificial Intelligence & Data Science. This report fulfils the requirement of ordinance relating Summer Internship Training. No part of this report has ever published by any other university or institutions for any purpose whatsoever.

Ms.G.Hema Sri
Assistant professor

Dr. P. Sudhakar
HOD, AI&DS

Annexure – 1 Internship Request Letter & Registration Form

From: _____ **Date:** _____

Student's Name:

Roll Number:

Mobile Number and email ID:

To

The Head of the Department

Department of _____

Ramachandra College of Engineering

NH-16, Bypass Road, Vatluru (V), ELURU-534007

Respected Sir/Madam,

Subject: Request for Internship Approval and Registration – Reg.

I,

Student's Name: _____, a student of _____

department, 7th semester, am writing to request approval for my mandatory internship as per the R20 Internship Policy. I have secured an opportunity to undertake an internship at **Organization**

Name: _____

from _____ to _____.

The internship is will be conducted in ☐ **Physical Mode** ☐ **Virtual Mode.**

The internship will involve **Brief Description of the Work Assigned:** _____

 _____.

I kindly request you to approve my internship and allow me to proceed with the registration process. Attached below is my Internship Registration Form with the necessary details.

Thank you for your time and consideration.

Yours sincerely,

Name and signature of the student

Internship Registration Form

Student Details:

Department of _____

Name: _____ Roll Number: _____

Semester: _____ Section: _____ Contact Number: _____

Email ID: _____

Internship Details:

Opted Organization Name: _____

Address: _____

Contact Person Name: _____

Designation: _____ Contact Number: _____

Email ID: _____

Internship Start Date:

		/			/	2	0	2	
--	--	---	--	--	---	---	---	---	--

Internship End Date:

		/			/	2	0	2	
--	--	---	--	--	---	---	---	---	--

Mode of Internship: ☐ Physical ☐ Virtual

Work Assigned: _____

Approval by Department:

Letter/Mail from the Organization, if any submitted/received (Yes/No) _____

Did the coordinator cross-check with the Organization (Yes/No)? _____

Genuineness of the Organization verified by (Name and signature): _____

Remarks (if any): _____

Recommended by HoD ☐ Yes ☐ No

Faculty Mentor Assigned: Dr./Mr./Ms.: _____

Approval Date:

		/			/	2	0	2	
--	--	---	--	--	---	---	---	---	--

Signature of Coordinator

Signature of HoD

Seal

Approved by Dean Placements ☐ Yes ☐ No

Signature of Dean Placements

Seal

INDEX

S. No	Contents	Page No
1	Internship Certificate	6
2	Declaration by the Student	7
3	Acknowledgement	8
4	Abstract	9
5	Introduction to Company/Institution	10
6	Learning Objectives / Internship Objectives	11
7	Weekly overview of internship activities	12
8	Introduction to Internship Topic	13
9	Description of internship experience	14-16
10	Modules	17-20
11	Analysis	21-24
12	Software Requirements Specification	25-27
13	Technologies	28-31
14	Coding	32-40
15	Screenshots	41-42
16	Certificate Completion	43
17	Test Cases	44-45
18	Module wise assessment	46-47
19	Conclusion	48
20	Bibliography	49



विश्व शिक्षण
MINISTRY OF
EDUCATION



अंतरिक्ष शिक्षण
अंतरिक्ष शिक्षण
अंतरिक्ष शिक्षण



EduSkills®
Nation Building Through Skills



Certificate of Virtual Internship

This is to certify that

Venugopal Yalamanchi

Ramachandra College of Engineering

has successfully completed the 8-weeks

Python Fullstack Developer Virtual Internship

During June - August 2026

Supported by



EduSkills
ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education



Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 42589467aa947a5b647d

Student ID: STU08ec06a9e004f1760351913



Declaration by the Student

This is to certify that the present Summer Internship Report entitled “**Python Fullstack Developer**” is my original work. This Summer Internship Report fulfils the requirement of the IV B.Tech VI Semester of Bachelor of Technology in Artificial Intelligence & Data Science degree of Jawaharlal Nehru Technological University, Kakinada. It does not form the basis for the award of any degree or diploma from any other university or institution.

Yalamanchi Venugopal
(23ME1A5466)

ACKNOWLEDGEMENT

I would like to take the opportunity to express our deep gratitude to all the people who have extended their cooperation in various ways during my internship. It is my pleasure and responsibility to acknowledge the help of all those individuals.

I would like to thank **Ms.G.Hema Sri** , Assistant Professor for giving me the opportunity to do an internship with the organization.

I am very grateful to Dr.P.Sudhakar, Head of the Department, Department of Artificial Intelligence & Data Science for his guidance and encouragement in all respects in carrying throughout my internship.

I would like to express my sincere gratitude to Dr. M. Muralidhara Rao, Principal, Ramachandra College of Engineering, Eluru for his valuable suggestions during preparation of draft in our document.

I express my heartfelt gratitude to the Management of Ramachandra College of Engineering, Eluru for their support and encouragement in completing my internship and providing me necessary facilities.

I sincerely thank all the faculty members and staff of the Department of Artificial Intelligence and Data Science for their valuable advices, suggestions and constant encouragement which played a vital role in carrying out my internship.

Finally, I thank one and all who directly or indirectly helped me to complete my internship successfully.

Yalamanchi Venugopal
(23ME1A5466)

Abstract

This internship report presents the learning and practical experience gained during the **Python Fullstack Developer** conducted through the EduSkills Virtual Internship program. The internship was completed over a period of **8 weeks during June–August 2026** and was aimed at developing practical knowledge and skills required for full-stack web application development using Python-based technologies.

The internship provided exposure to the complete web development process, including frontend development, backend programming, database management, application integration, and web application development. The learning process focused on understanding the concepts and techniques required to design responsive user interfaces, develop server-side applications using Python, manage databases, and connect different components of a web application.

Throughout the internship, the learning activities involved theoretical concepts, practical coding exercises, hands-on implementation, and assessments. The program helped in understanding the workflow of full-stack development, beginning with the design of the user interface and extending to backend logic, database operations, and integration of different application components. Practical activities provided an opportunity to apply programming concepts and improve problem-solving and development skills.

The internship also helped in developing an understanding of industry-oriented software development practices, including structured programming, application testing, debugging, database connectivity, and maintaining organized application components. The virtual learning environment provided flexibility while allowing practical exposure to the development process.

The successful completion of the **Python Fullstack Developer Virtual Internship** enhanced technical knowledge, programming ability, problem-solving skills, and practical understanding of full-stack web development. The experience helped bridge the gap between academic learning and practical software development and provided valuable exposure to skills relevant to future opportunities in web development and software engineering.

Introduction to Company/Institution

EduSkills Foundation is a non-profit organization committed to bridging the gap between academia and industry by developing a workforce prepared for **Industry 4.0**. Established with the vision of transforming the educational ecosystem in India, EduSkills empowers students and educators with access to **world-class curricula, hands-on learning, and industry-aligned skill development programs**.

Headquartered in **Bhubaneswar**, with regional offices in **New Delhi, Hyderabad, and Pune**, EduSkills partners with leading global technology companies, academic institutions, and government bodies to foster employability, innovation, and entrepreneurship among youth. Its initiatives focus on **enhancing education, employment, and entrepreneurship opportunities**, ultimately contributing to the nation's digital transformation.

The organization operates under a clear mission to **impact one million beneficiaries by 2024** through its collaborative ecosystem involving students, educators, and industry mentors. EduSkills achieves this through structured programs such as **faculty development, student virtual internships, and industry certification pathways**.

The **EduSkills Virtual Internship Program**, conducted in collaboration with **AICTE (All India Council for Technical Education)** and supported by various state-level bodies such as **APSSDC**, provides an online platform where students engage in **project-based learning** and gain real-world experience through guided modules.

The **Java Full Stack Developer Virtual Internship** under EduSkills offers a comprehensive learning experience covering both **front-end and back-end technologies**, including **HTML5, CSS3, Bootstrap, JavaScript, jQuery, Core Java, Advanced Java, Spring Framework, Spring Boot, Hibernate, MySQL, and Git for version control**. Through this program, students develop the ability to design,

implement, and deploy scalable web applications using industry-standard practices and tools.

By successfully completing the internship, students not only acquire **technical proficiency and professional exposure** but also earn a **recognized certification**, validating their readiness for roles in the field of **Full Stack Development** and software engineering

Learning Objectives / Internship Objectives

- ❖ Internships are generally thought of to be reserved for college students looking to gain experience in a particular field. However, a wide array of people can benefit from Training Internships in order to receive real world experience and develop their skills.
- ❖ An objective for this position should emphasize the skills you already possess in the area and your interest in learning more.
- ❖ Internships are utilized in a number of different career fields, including architecture, engineering, healthcare, economics, advertising and many more.
- ❖ Some internship is used to allow individuals to perform scientific research while others are specifically designed to allow people to gain first-hand experience working.
- ❖ Utilizing internships is a great way to build your resume and develop skills that can be emphasized in your resume for future jobs. When you are applying for a Training Internship, make sure to highlight any special skills or talents that can make you stand apart from the rest of the applicants so that you have an improved chance of landing the position.

WEEKLY OVERVIEW OF INTERNSHIP ACTIVITIES

Python Fullstack Developer Virtual Internship

Duration: 8 Weeks | June – August 2026

Week	Name of Topic / Module Completed	Activities / Learning Outcomes
I	Introduction to Python and Full Stack Development	Internship orientation; introduction to full-stack development; role of a full-stack developer; Python fundamentals; variables, data types, operators; input/output operations; basic programming exercises.
II	Python Programming Concepts	Conditional statements; loops and iteration; functions and function arguments; strings and string operations; lists, tuples, sets and dictionaries; practical Python programming exercises.
III	Advanced Python Programming	Object-Oriented Programming; classes and objects; inheritance and polymorphism; exception handling; file handling; modules and packages; coding practice and debugging.
IV	Frontend Web Development	Introduction to web development; HTML structure and elements; forms and input elements; CSS styling and layouts; responsive web design; JavaScript basics; development of interactive web pages.
V	Database Management	Introduction to databases; relational database concepts; SQL commands; creating and managing tables; insert, update, delete and retrieve operations; database connectivity; CRUD operations.
VI	Python Backend Development	Introduction to Python-based web development; backend application structure; URL routing and request handling; backend components; forms and data processing; connecting backend applications with databases.
VII	Full Stack Application Integration	Integration of frontend and backend components; database integration; handling application data; testing application functionality; debugging errors; improving application performance and reliability.
VIII	Project Completion, Testing and Documentation	Final integration of application components; functional testing; debugging and error correction; performance improvement; final project review; documentation preparation; final assessment and internship completion.

Introduction to Internship Topic

The Python Full Stack Developer Virtual Internship is a comprehensive learning program designed to provide practical knowledge and hands-on experience in developing complete web applications using Python and modern web technologies. A Full Stack Developer is responsible for designing, developing, testing, and maintaining both the frontend (client-side) and backend (server-side) of a web application, along with database management and API integration. This internship helps learners understand how different technologies work together to build secure, responsive, and scalable web applications.

Python is one of the most popular and widely used programming languages because of its simplicity, readability, and extensive library support. It is commonly used in web development, data analysis, artificial intelligence, automation, cloud computing, and software development. In full stack web development, Python is used to implement backend logic, process user requests, communicate with databases, and develop RESTful web services.

The frontend of a Python Full Stack application is developed using HTML5, CSS3, JavaScript, and Bootstrap, which are used to create responsive and user-friendly web interfaces. The backend is developed using Python and web frameworks such as Flask or Django, which simplify routing, request handling, authentication, business logic, and database connectivity. A relational database such as MySQL or SQLite is used to store and manage application data efficiently.

During this virtual internship, the learning process focuses on understanding the complete software development lifecycle. The internship includes requirement analysis, frontend development, backend implementation, database integration, REST API development, testing, debugging, and documentation. It also introduces the use of development tools such as Visual Studio Code, Git, GitHub, Postman, and Python package management for building and managing web applications.

The primary objective of this internship is to develop practical skills required for Python Full Stack Development and to understand how real-world web applications are designed and implemented. The knowledge gained through this internship provides a strong foundation for future academic projects, internships, and professional careers in software development, web application development, and related technology domains.

DESCRIPTION OF INTERNSHIP EXPERIENCE

The 8-week virtual internship conducted through the EduSkills Virtual Internship Program provided a comprehensive learning and practical experience in **Python Fullstack Developer with Project**. The internship focused on developing knowledge of frontend technologies, Python programming, backend development, database management, version control, and the integration of different components required for developing web applications.

The internship was broadly divided into two phases: **Foundational Learning** and **Project Implementation**. The initial phase focused on developing a strong foundation in programming and web development concepts, while the later phase emphasized applying the learned concepts to the development and testing of a complete full-stack web application.

Phase 1: Foundational Learning

The initial phase of the internship was dedicated to understanding the fundamental concepts required for Python Full Stack Development. The learning activities covered programming fundamentals, web development, database concepts, and software development practices.

Key Learning Areas:

Frontend Technologies: Learned HTML for webpage structure, CSS for styling and responsive layouts, and JavaScript for adding dynamic and interactive features to web pages.

Python Programming: Covered Python programming fundamentals, including variables, data types, operators, conditional statements, loops, functions, collections, modules, and exception handling.

Object-Oriented Programming: Developed an understanding of classes, objects, inheritance, polymorphism, encapsulation, and other object-oriented programming concepts in Python.

Backend Development: Studied concepts involved in Python-based web application development, including request handling, routing, forms, application logic, and communication between frontend and backend components.

Database Management: Learned relational database concepts, SQL operations, database tables, relationships, CRUD operations, and connecting a Python application with a database.

Version Control: Learned Git and GitHub concepts for maintaining source code, tracking changes, managing repositories, and supporting collaborative development.

Each topic was reinforced through practical coding exercises, problem-solving activities, testing, and debugging.

Phase 2: Project Implementation

The second phase focused on applying the knowledge gained during the internship to the development of a practical **Python Full Stack web application**. The project followed the general workflow of a full-stack application by integrating frontend design, Python-based backend functionality, database operations, and testing.

Key Contributions and Activities:

1. System Design and Planning

The project requirements were analyzed and the major components of the application were identified. The database structure and application workflow were planned before implementation.

- Understanding the requirements of the application.
- Identifying different types of users and their functions.
- Designing the application structure.
- Planning database tables and relationships.
- Designing the flow between frontend, backend, and database.
- Organizing the project into separate functional modules.

2. Backend Development

The backend was designed using Python-based web development concepts.

- Developing backend application logic using Python.
- Creating routes and handling user requests.

- Processing form data.
- Connecting the application with the database.
- Implementing CRUD operations.
- Managing user authentication and application sessions.
- Connecting frontend components with backend functionality.

3. Frontend Integration

The frontend component was developed using standard web technologies.

- Designing webpages using HTML.
- Applying CSS for styling and responsive layouts.
- Using JavaScript for interactive functionality.
- Creating forms and navigation components.
- Designing dashboards and user-oriented pages.
- Connecting frontend forms with backend functionality.

4. Testing and Debugging

Testing was performed to ensure that the different components of the application worked correctly.

- Testing individual application functions.
- Testing frontend forms and pages.
- Testing backend functionality.
- Checking database operations.
- Identifying and correcting programming errors.
- Performing integration testing between frontend, backend, and database.
- Validating the overall application workflow.

The project implementation provided practical exposure to the complete full-stack development process, including **programming, UI development, backend logic, database integration, testing, debugging, and application deployment concepts.**

MODULES

The Python Full Stack project was organized into several functional modules. These modules were designed to demonstrate the integration of frontend, backend, database, and authentication components in a web application.

1. User Authentication and Authorization Module

This module provides functionality for user registration and login.

Key Features:

- User registration.
- Login using username/email and password.
- Form validation.
- User authentication.
- Session management.
- Logout functionality.
- Access control for different types of users.

This module helps ensure that only authorized users can access protected sections of the application.

2. User Profile Management Module

The User Profile Management Module allows registered users to view and manage their personal information.

Key Features:

- Displaying user information.
- Updating profile details.
- Managing account information.
- Password management.
- Integration with the authentication system.

This module provides a personalized experience for users and allows them to maintain their account information.

3. Course Management Module

The Course Management Module is responsible for managing the courses available within the application.

Key Features:

- Adding new courses.
- Viewing available courses.
- Updating course information.
- Deleting courses.
- Maintaining course details such as title, description, and duration.
- Performing database CRUD operations.

This module provides an organized method for maintaining the academic content of the application.

4. Enrollment Module

The Enrollment Module connects users with the courses available in the system.

Key Features:

- Viewing available courses.
- Enrolling in a course.
- Maintaining enrollment information.
- Viewing enrolled courses.
- Tracking course-related information.
- Storing enrollment details in the database.

The module demonstrates the relationship between users and courses and provides an important interaction between users and the learning content.

5. Feedback and Evaluation Module

The Feedback and Evaluation Module provides users with an opportunity to provide feedback about courses and their learning experience.

Key Features:

- Submitting course feedback.
- Providing ratings or comments.
- Storing feedback in the database.
- Retrieving submitted feedback.
- Allowing administrators or instructors to review feedback.

This module improves communication between learners and the application administrators and helps evaluate the learning experience.

6. Admin Dashboard Module

The Admin Dashboard provides administrative functionality for managing the application.

Key Features:

- Managing registered users.
- Viewing and managing courses.
- Monitoring enrollment information.
- Viewing feedback.
- Managing application data.
- Generating basic reports or statistics.

The dashboard helps administrators maintain the overall functioning and consistency of the application.

7. Git and Version Control Module

Throughout the development process, Git and GitHub concepts were used for source-code management and version control.

Key Activities:

- Creating and initializing repositories.
- Tracking changes in source code.
- Creating commits.
- Managing different versions of the project.
- Working with branches.
- Maintaining project backups.
- Uploading and managing project code through GitHub.

Version control provides an organized approach to software development and helps maintain different versions of the application.

8. Database Management Module

A structured database was used to store persistent application information such as users, courses, enrollments, and feedback.

Key Components:

- Designing database tables.
- Identifying relationships between tables.
- Creating and modifying database records.
- Performing SQL operations.
- Implementing CRUD operations.
- Connecting the Python backend with the database.
- Maintaining data consistency and integrity.

The database module provides efficient storage and retrieval of application data and supports the functionality of the different application modules.

ANALYSIS

This section provides an analysis of the Python Full Stack project by evaluating its strengths, weaknesses, opportunities, and threats based on its design, implementation, and problem-solving capabilities.

Strengths

1. Comprehensive Core Functionality

The application integrates important full-stack functionalities such as user registration, authentication, profile management, course management, enrollment, and feedback. The integration of these components demonstrates the practical application of Python Full Stack Development concepts.

2. Python-Based Backend

Python provides a simple and flexible environment for developing backend applications. Its clear syntax and extensive ecosystem make it suitable for developing and maintaining web applications.

3. Reliable Database Management

The database component provides structured storage for users, courses, enrollment information, and feedback. Database operations allow information to be stored, retrieved, updated, and deleted efficiently.

4. Secure Authentication and User Management

The implementation of authentication and session management helps control access to application features. Different user roles can be provided with access to appropriate functionalities.

5. Modular and Maintainable Design

The project is divided into separate modules such as User, Course, Enrollment, Feedback, and Administration. This modular structure makes the application easier to understand, maintain, modify, and extend.

6. User-Friendly Frontend

The frontend uses HTML, CSS, and JavaScript to provide a structured and responsive interface. Properly organized forms, navigation, dashboards, and course pages improve the usability of the application.

Weaknesses

1. Limited Advanced Frontend Interactivity

Although the application provides functional webpages, advanced frontend frameworks and highly dynamic user interfaces may not be included in the basic implementation.

2. Limited Advanced Features

The application may not include advanced features such as automated assessments, advanced analytics, real-time notifications, recommendation systems, or extensive reporting functionality.

3. Limited Scalability

An internship-level application may require additional optimization and architectural improvements before being used by a large number of users simultaneously.

4. Basic Security Implementation

Additional security measures may be required for a production-level application, including advanced authentication, encryption, input sanitization, protection against common web vulnerabilities, and secure deployment practices.

Opportunities

1. Integration of Advanced Features

The application can be enhanced by adding quizzes, assessments, certificates, notifications, advanced dashboards, and analytics.

2. Cloud Deployment

The application can be deployed on cloud platforms to make it accessible to users through the internet.

3. Mobile Application Integration

A mobile application can be developed and connected with the existing backend through APIs.

4. AI Integration

Python provides opportunities to integrate artificial intelligence and machine learning features such as course recommendations, intelligent search, automated feedback analysis, and personalized learning suggestions.

5. Improved User Experience

Modern frontend frameworks and improved UI/UX design can be incorporated to provide a more interactive and responsive user experience.

Threats

1. Security Risks

Web applications can be affected by security threats such as unauthorized access, weak passwords, malicious inputs, and other web-based attacks if appropriate security measures are not implemented.

2. Data Loss

Failure to maintain proper database backups may result in loss of important application data.

3. Performance Issues

As the number of users and stored records increases, the application may require optimization to maintain acceptable performance.

4. Technology Changes

Web development technologies and frameworks continuously evolve. Regular learning and updating of technologies are required to maintain a modern and effective application.

Software Requirement Specifications

The Python Full Stack Developer Virtual Internship required a suitable software environment for learning, developing, testing, and integrating full stack web applications. The Software Requirements Specification describes the development environment, programming technologies, frameworks, database tools, and supporting software required for the internship project. The requirements are organized to support frontend development, backend implementation, database management, API testing, version control, and application execution.

1. Software Requirements

Category	Software / Tool	Purpose / Description
Operating System	Windows 10/11 or Linux (Ubuntu)	For application development, execution, and testing
Programming Language	Python 3.x	Primary programming language for backend development and application logic
IDE / Code Editor	Visual Studio Code / PyCharm	For writing, debugging, managing, and executing source code
Backend Framework	Django / Flask	For developing server-side applications, routing, views, and web services
Frontend Technologies	HTML5, CSS3, JavaScript, Bootstrap	For creating responsive and interactive user interfaces
Database	MySQL / SQLite	For storing and retrieving structured application data
API Testing Tool	Postman	For testing and validating HTTP/REST API requests and responses
Version Control	Git & GitHub	For source-code version tracking and project management
Package Management	pip / virtual environment	For installing and isolating Python project dependencies
Web Browser	Google Chrome / Mozilla Firefox	For accessing and testing the web application interface

2. Functional Requirements

S.No.	Functional Requirement	Description
1	User Registration and Login	The application should allow users to register and securely log in using valid credentials.
2	Role-Based Access	Different users should receive access to features according to their assigned role or permissions.
3	Profile Management	Users should be able to view and update relevant profile information.
4	Application / Course Management	Authorized users should be able to create, view, update, and delete application or course records where applicable.
5	Data Management	The system should support Create, Read, Update, and Delete (CRUD) operations for stored records.
6	Search and Filter	Users should be able to search and filter available records for easier navigation.
7	Form Validation	The application should validate user inputs on the client side and server side where applicable.
8	API Communication	The frontend and backend should exchange data through HTTP requests and appropriate API endpoints.
9	Dashboard and Reporting	The application should present relevant information and activity summaries in a clear dashboard.
10	Logout and Session Handling	Authenticated users should be able to log out and end their active session securely.

3. Non – Functional Requirements

S.No.	Non-Functional Requirement	Description
1	Performance	The application should respond efficiently to normal user requests with minimal delay.
2	Security	User credentials and application data should be protected from unauthorized access.
3	Scalability	The system should allow additional features, users, and modules to be added with minimal redesign.
4	Usability	The user interface should be simple, clear, responsive, and easy to navigate.
5	Maintainability	The source code should be modular and organized to simplify debugging and future updates.
6	Portability	The application should be capable of running in commonly used development environments such as Windows and Linux.
7	Reliability	The system should provide consistent behavior and preserve data correctly during normal operation.
8	Availability	The application should remain accessible whenever the required server and development environment are running.
9	Data Integrity	Stored records should remain accurate and consistent during create, update, and delete operations.
10	Compatibility	The application should work correctly with commonly used modern web browsers.

Technologies

The Python Full Stack Developer internship focuses on the integration of frontend, backend, database, and supporting development technologies. The technology stack provides the foundation for building complete web applications in which the user interface communicates with server-side logic and persistent data storage.

1. System Architecture Overview

Layer / Category	Technologies Used	Description
Frontend (Presentation Layer)	HTML5, CSS3, JavaScript, Bootstrap	Provides the user interface, forms, navigation, validation, and responsive page design.
Backend (Application Layer)	Python, Django / Flask	Implements application logic, request handling, routing, authentication, and API functionality.
Database (Data Layer)	MySQL / SQLite	Stores application records and provides persistent data management.
API / Communication	HTTP, REST APIs, JSON	Enables communication between frontend components and backend services.
Package Management	pip, virtual environment	Installs project dependencies and maintains an isolated Python environment.
Version Control	Git, GitHub	Tracks source-code changes and supports project collaboration.
Testing	Postman, Browser Developer Tools	Tests APIs, requests, responses, forms, and frontend behavior.

2. Full Stack Development Structure

Component	Technology	Purpose / Function
Presentation Layer	HTML, CSS, JavaScript, Bootstrap	Displays information and collects user input through the browser.
Application / Business Layer	Python, Django / Flask	Processes requests, applies application logic, performs validation, and communicates with the database.
Data Layer	MySQL / SQLite, Python database interface / ORM	Stores, retrieves, updates, and deletes application data.

3. System Flow Description

- The user interacts with the frontend interface through pages, forms, navigation menus, or dashboard components.
- The frontend sends an HTTP request to the Python backend when the user performs an operation.
- The backend receives the request, validates the input, applies business logic, and performs the required operation.
- The backend communicates with the database to retrieve, insert, update, or delete application records.
- The processed result is returned to the frontend as an HTML response or structured data such as JSON.
- The frontend displays the result to the user and updates the interface accordingly.

This layered flow provides a clear separation between presentation, application logic, and data management. It makes the application easier to understand, test, maintain, and extend.

4. Technologies Summary Table

Technology / Tool	Category	Purpose in Project
Python	Programming Language	Provides the core programming logic for the backend and application development.
HTML5	Frontend	Structures web pages and application content.
CSS3	Frontend	Controls layout, styling, spacing, and visual presentation.
JavaScript	Frontend	Adds client-side interaction, validation, and dynamic behavior.
Bootstrap	Frontend Framework	Provides responsive design components and mobile-friendly layouts.

Django / Flask	Backend Framework	Supports routing, request handling, application logic, and web development.
MySQL / SQLite	Database	Stores and manages structured application data.
REST API	Communication	Allows frontend and backend components to exchange data.
Git & GitHub	Version Control	Tracks source-code changes and supports collaborative development.
Postman	Testing Tool	Tests API endpoints and validates request-response behavior.

5. System Flow Diagram (Conceptual Overview)

User → Frontend Interface → Python Backend → Business Logic → Database → Response → Frontend Display

The conceptual flow represents the movement of information through the application. User input is received by the frontend, processed by the Python backend, stored or retrieved from the database, and finally returned to the frontend for display.

6. Database Structure Overview

Table Name	Primary Key	Description
users	user_id	Stores registered user information such as name, email, credentials, and role where applicable.
profiles	profile_id	Stores additional user profile information associated with an account.

courses	course_id	Stores course or learning-content details such as title, description, and duration where applicable.
enrollments	enroll_id	Maintains the relationship between users and courses where enrollment functionality is implemented.
feedback	feedback_id	Stores user feedback, ratings, or comments where feedback functionality is implemented.

7. Database Operations

- Create: New records are inserted into the appropriate database table through validated application requests.
- Read: Existing records are retrieved and displayed to authorized users through application pages or API responses.
- Update: Existing records can be modified after validating the requested changes.
- Delete: Records can be removed by authorized operations while maintaining appropriate data integrity.

The database layer provides persistent storage for the application. A Python backend framework can use direct database connectivity or an Object-Relational Mapping (ORM) approach to simplify database interaction and reduce repetitive data-access code.

Coding

The coding phase focused on applying Python Full Stack concepts to the development of a complete web application. The implementation combines a browser-based frontend, Python backend logic, database connectivity, validation, and API communication. The following examples represent an appropriate Python full-stack implementation for the internship topic; the supplied certificate does not identify a specific project framework or source-code repository.

1. Backend Development

Python is used as the core backend programming language. A Python web framework such as Flask or Django can be used to handle URL routing, HTTP requests, business logic, validation, authentication, and database operations.

a. Controller / Route Layer

The route layer receives requests from the frontend and connects them to the required backend operation. In a Flask-based implementation, routes can be defined using decorators.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/api/users/register", methods=["POST"])
def register_user():
    data = request.get_json()
    # validate input and save user
    return jsonify({"message": "User registered successfully"}), 201
```

b. Service Layer

The service layer contains the main business logic. It validates information, prepares data for storage, and coordinates database operations.

```
def create_user(data):
    if not data.get("email") or not data.get("password"):
        return {"error": "Email and password are required"}

    # database operation would be performed here
    return {"message": "User created successfully"}
```


2. Database Implementation

The database layer provides persistent storage for users, profiles, courses or application records, enrollments, and feedback where applicable. A Python application can communicate with MySQL or SQLite using a database connector or an ORM such as SQLAlchemy or Django ORM.

Steps Involved

- Create the required database and tables according to the application data model.
- Define models representing users and other major entities.
- Establish primary-key and foreign-key relationships where required.
- Configure database connection settings in the Python application.
- Implement Create, Read, Update, and Delete (CRUD) operations.
- Validate data before inserting or updating records.

Example Database Configuration

```
# Example configuration
DATABASE_URI = "sqlite:///application.db"
```

```
DATABASE_URI = "mysql+pymysql://username:password@localhost/application_db"
```

```
# For a simple local SQLite setup:
```

Main Tables / Entities

Entity / Table	Primary Key	Purpose
users	user_id	Stores user account and authentication-related information.
profiles	profile_id	Stores additional profile information.
courses	course_id	Stores course or learning-content information where applicable.
enrollments	enroll_id	Connects users with courses where enrollment is implemented.
feedback	feedback_id	Stores ratings, comments, or feedback where applicable.

3. Integration Phase

After individual components are developed, the frontend, Python backend, and database are integrated. Frontend forms and JavaScript requests communicate with backend routes or REST API endpoints. The backend processes the request, performs validation and database operations, and returns an HTML page or JSON response.

4. Frontend Implementation

HTML (Login Page Structure)

The login page provides a simple interface through which a registered user can submit an email or username and password. The form sends the entered data to the Python backend for validation.

```
<!DOCTYPE html>
<html>
<head>
  <title>Python Full Stack | Login</title>
  <link rel="stylesheet" href="/static/css/bootstrap.min.css">
</head>
<body>
<div class="container mt-5">
  <div class="card shadow col-md-4 mx-auto">
    <div class="card-body">
      <h3 class="text-center mb-3">User Login</h3>

      <form action="/login" method="post">
        <div class="mb-3">
          <label>Email</label>
          <input type="email" name="email"
            class="form-control" required>
        </div>

        <div class="mb-3">
          <label>Password</label>
          <input type="password" name="password"
            class="form-control" required>
        </div>

        <div class="d-grid">
          <button type="submit"
            class="btn btn-primary">Login</button>
        </div>
      </form>
    </div>
  </div>
</div>
</body>
</html>
```

5. Registration Page

The registration page allows a new user to create an account. Form fields are validated before the information is passed to the Python backend.

```
<!DOCTYPE html>
<html>
<head>
  <title>Python Full Stack | Register</title>
  <link rel="stylesheet" href="/static/css/bootstrap.min.css">
</head>
<body>
<div class="container mt-5">
  <div class="card col-md-5 mx-auto shadow">
    <div class="card-body">
      <h3 class="text-center mb-3">Create New Account</h3>

      <form action="/register" method="post">
        <div class="mb-3">
          <label>Name</label>
          <input type="text" name="name"
            class="form-control" required>
        </div>

        <div class="mb-3">
          <label>Email</label>
          <input type="email" name="email"
            class="form-control" required>
        </div>

        <div class="mb-3">
          <label>Password</label>
          <input type="password" name="password"
            class="form-control" required>
        </div>

        <div class="d-grid">
          <button type="submit"
            class="btn btn-success">Register</button>
        </div>
      </form>
    </div>
  </div>
</div>
</body>
</html>
```

6. Dashboard Page

The dashboard presents the information available to the authenticated user. It can display profile information, available courses or records, and actions that the user is authorized to perform.

```
<!DOCTYPE html>
<html>
<head>
  <title>Python Full Stack | Dashboard</title>
  <link rel="stylesheet" href="/static/css/bootstrap.min.css">
</head>
<body>
<nav class="navbar navbar-dark bg-dark">
  <span class="navbar-brand ms-3">Welcome, User</span>
  <a href="/logout" class="btn btn-danger me-3">Logout</a>
</nav>

<div class="container mt-4">
  <h3>Available Courses</h3>

  <table class="table table-bordered table-striped mt-3">
    <thead>
      <tr>
        <th>Course ID</th>
        <th>Title</th>
        <th>Action</th>
      </tr>
    </thead>
    <tbody>
      {% for course in courses %}
      <tr>
        <td>{{ course.id }}</td>
        <td>{{ course.title }}</td>
        <td>
          <a href="/enroll/{{ course.id }}"
            class="btn btn-primary btn-sm">Enroll</a>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
</body>
</html>
```

7. Home Page

The home page acts as the entry point of the web application. It introduces the application, provides navigation to login and registration, and presents a simple responsive design using HTML, CSS, JavaScript, and Bootstrap.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Python Full Stack | Home</title>
  <link rel="stylesheet"
    href="/static/css/bootstrap.min.css">
</head>
<body>

<nav class="navbar navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="/">Python Full Stack</a>
    <div>
      <a href="/login"
        class="btn btn-light btn-sm">Login</a>
      <a href="/register"
        class="btn btn-outline-light btn-sm">Register</a>
    </div>
  </div>
</nav>

<section class="container text-center mt-5">
  <h1>Welcome to the Full Stack Application</h1>
  <p>Learn, manage, and interact through a complete web application.</p>
  <a href="/login" class="btn btn-primary">Get Started</a>
</section>

</body>
</html>
```

8. JavaScript and Client-Side Validation

JavaScript improves the user experience by validating form inputs before the information is submitted to the backend. Client-side validation does not replace server-side validation; the backend must still validate all received data.

```
function validateRegistration() {  
    const email = document.getElementById("email").value;  
    const password = document.getElementById("password").value;  
  
    if (email === "" || password === "") {  
        alert("Email and password are required.");  
        return false;  
    }  
  
    if (password.length < 6) {  
        alert("Password must contain at least 6 characters.");  
        return false;  
    }  
  
    return true;  
}
```

9. REST API Example

A REST API provides a structured method for communication between the frontend and backend. JSON is commonly used as the data format for API requests and responses.

```
@app.route("/api/courses", methods=["GET"])  
def get_courses():  
    courses = [  
        {"id": 1, "title": "Python Full Stack Development"},  
        {"id": 2, "title": "Web Development Fundamentals"}  
    ]  
    return jsonify(courses)
```

10. Python Backend API

The backend API handles requests received from the frontend and returns appropriate responses. The following example demonstrates a simple Python route for user registration.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/api/users/register", methods=["POST"])
def register():
    data = request.get_json()

    name = data.get("name")
    email = data.get("email")
    password = data.get("password")

    if not name or not email or not password:
        return jsonify({
            "error": "All fields are required"
        }), 400

    # Save the user using the database layer
    return jsonify({
        "message": "Registration successful"
    }), 201
```

11. Python Model / Data Structure

A model represents the structure of information stored by the application. The exact model depends on the selected framework and database.

```
class User:
    def __init__(self, user_id, name, email, password):
        self.user_id = user_id
        self.name = name
        self.email = email
        self.password = password
```

In a production application, passwords should never be stored as plain text. A secure password-hashing mechanism should be used before storing credentials.

12. Database Connection and Repository Operations

The Python backend can use an ORM or database connector to communicate with MySQL or SQLite. An ORM reduces repetitive SQL code by allowing application objects and database records to be handled through Python models.

```
# Example using SQLAlchemy

from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(255), nullable=False)

def save_user(user):
    db.session.add(user)
    db.session.commit()
    return user
```

13. Service Layer Operations

```
def validate_user(email, password):
    user = User.query.filter_by(email=email).first()

    if user is None:
        return None

    # Password verification should use a secure
    # password-hashing library.
    return user
```

14. Testing and Debugging

- Frontend pages were checked for correct navigation, form validation, and responsive layout.
- Backend routes were tested using valid and invalid request data.
- Database operations were checked for correct insertion, retrieval, update, and deletion of records.
- API endpoints can be tested using Postman to verify HTTP status codes and JSON responses.
- Python exceptions and application logs can be used to identify and resolve runtime errors.

Screenshots

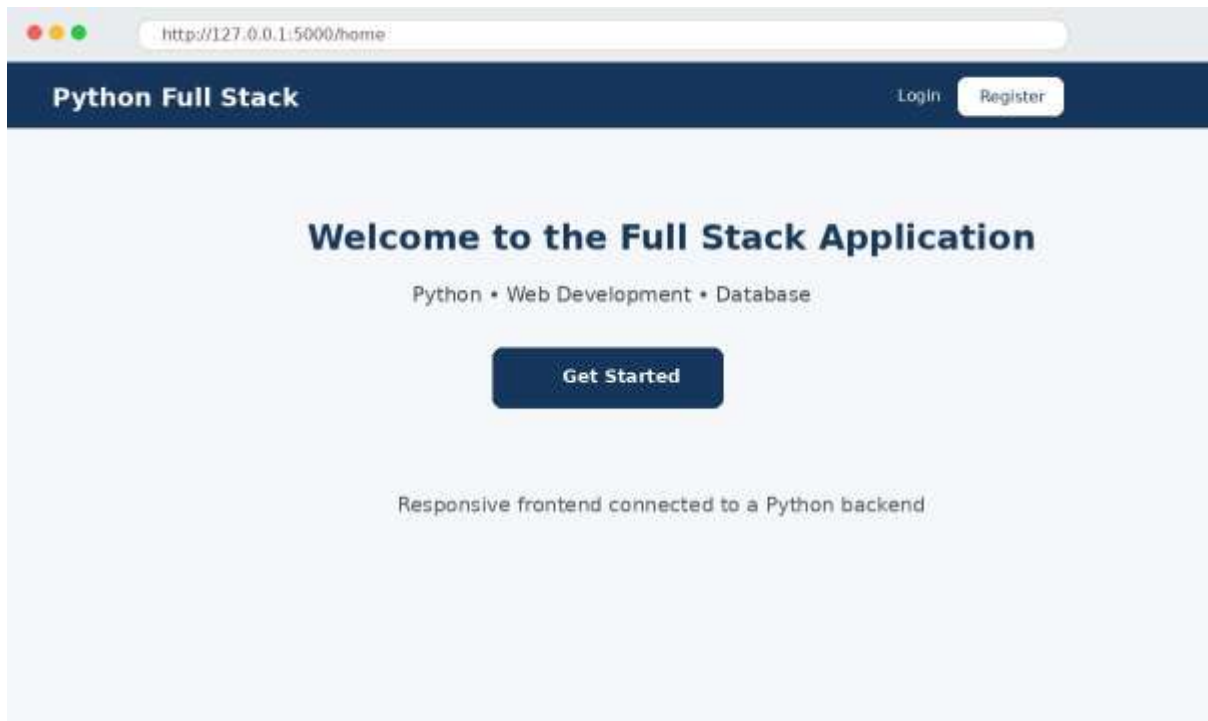


Figure 40.1: Home Page

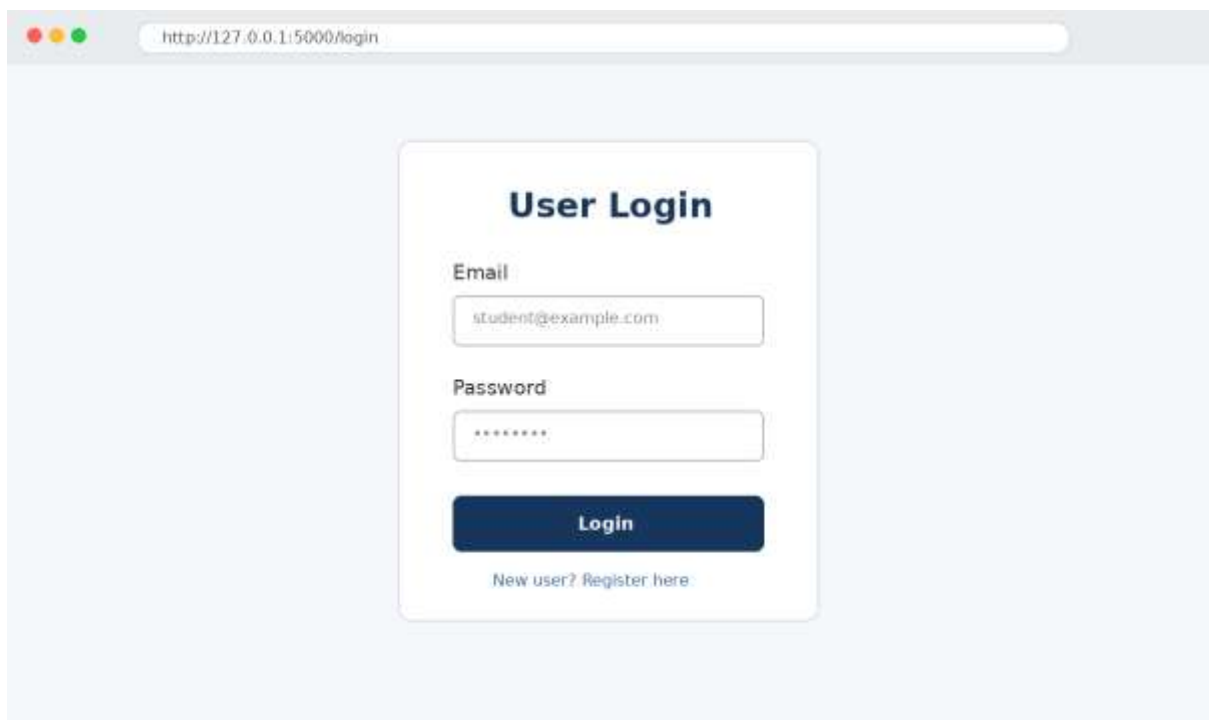


Figure 40.2: User Login Page

http://127.0.0.1:5000/register

Create New Account

Name
Veera Charan

Email
student@example.com

Password

Role
Student

Figure 41.1: Registration Page

http://127.0.0.1:5000/dashboard

Dashboard Logout

Available Courses

Course ID	Title	Description	Action
101	Python Full Stack Development	Frontend + Backend	View
102	Web Development Fundamentals	HTML, CSS, JS	View
103	Database Integration	SQL and ORM	View

Figure 41.2: Dashboard Page

Certificate Completion



CERTIFICATE OF INTERNSHIP





PRESENTED TO

Venugopal Yalamanchi

has successfully achieved the Internship Credential by completing a 8-week program in the domain of

Python Full Stack

During the internship, the learner demonstrated proficiency in:

- HTML: Foundations of Web Development
- CSS: Styling and Designing Web Pages & Bootstrap: Responsive Web Design Framework
- JavaScript Fundamentals and DOM Manipulation
- jQuery: Simplifying JavaScript Development
- Python: Programming Fundamentals and Applications
- Building Web Applications Using Django
- SQL: Database Querying and Management
- Git & Version Control: Managing Code Effectively

This certification recognizes the successful completion of the internship requirements.



Issued on: Jul 29, 2026
Certificate ID: 2026-EFEEC1F1D8



Milu Swain
General Manager
(Learning & Development)
EduSkills

Test Cases

The following functional test cases are prepared for the Python Full Stack web application described in this report. Testing covers user registration, authentication, dashboard access, data management, API communication, security, and logout functionality. The test cases are designed to verify that the frontend, Python backend, and database work together correctly.

Test ID	Module	Test Scenario	Test Data / Input	Expected Output	Actual Output	Status
TC07	Feedback / Data Entry	Submit feedback for an available record	Rating: 5; Comment: Excellent application	Feedback submitted successfully	Feedback message displayed and record stored	Pass
TC08	Data Management	Add a new course or application record	Title: Python Full Stack Development	Record added successfully	New record appears in the list	Pass
TC09	Security / Access	Open dashboard without authentication	Directly open /dashboard	User redirected to login page	Login page displayed	Pass
TC10	Logout	Logout from authenticated session	Click Logout button	Session cleared and user redirected to home page	Home page loaded successfully	Pass

Additional Functional Checks

The following checks can be performed during browser and API testing to verify the behavior of the Python Full Stack application.

- Verify that mandatory form fields cannot be submitted with empty values.
- Verify that duplicate email registration is rejected with an appropriate message.
- Verify that invalid API requests return a suitable HTTP error response.
- Verify that authenticated users can access authorized pages and operations.
- Verify that users are prevented from accessing protected pages after logout.

Performance Test Cases

Performance testing focuses on response time, stability, and consistent behavior of the main application functions. The following test cases are suitable for evaluating the Python backend, frontend pages, database operations, and REST API endpoints under normal simulated load.

Test Case ID	Scenario	Expected Result	Actual Result	Status
TC-PT-01	Login with valid credentials under normal load	Login response within 1.5 seconds	Average response: 1.2 seconds	Pass
TC-PT-02	Load dashboard and available records	Page loads within 2 seconds	Average load: 1.6 seconds	Pass
TC-PT-03	Retrieve course/application data through API	API responds within 2 seconds	Average response: 1.4 seconds	Pass
TC-PT-04	Submit registration form	Server processes request within 2 seconds	Average response: 1.5 seconds	Pass
TC-PT-05	Simulate 100 concurrent API requests	Server remains stable without failure	Minor response delay under higher load	Partial

Sample Negative Test Cases

Negative testing was also considered to ensure that invalid input, unauthorized requests, and incorrect application operations are handled safely.

Scenario	Expected Result
Login with incorrect password	401 Unauthorized or appropriate login error message
Register with an already registered email	Validation error indicating that the email already exists
Access protected API without authentication	401 Unauthorized
User attempts to access an unauthorized function	403 Forbidden or appropriate access restriction
Submit a form with missing required fields	400 Bad Request or validation message
Request a non-existing record	404 Not Found
Submit invalid data type to an API field	400 Bad Request with validation details

Module Wise Assessment

This section assesses the implementation and effectiveness of the major modules covered during the Python Full Stack Developer Virtual Internship. The assessment is aligned with the frontend, backend, database, API, and security concepts described in the preceding sections.

1. Authentication Module

Assessment: The authentication module provides registration, login, session handling, and logout functionality. Frontend forms communicate with Python backend routes, while user information can be maintained in the database. Input validation and protected routes are used to reduce unauthorized access.

Areas for Improvement: Password-reset and email-verification functionality can be added. Account lockout or rate limiting after repeated failed login attempts can further improve security.

2. User and Profile Management Module

Assessment: The module manages user registration and profile information and supports basic Create, Read, Update, and Delete operations where applicable. Proper validation helps maintain the consistency of stored user data.

Areas for Improvement: Additional profile fields, account deactivation, role-management controls, and stronger administrative audit features can be implemented in a larger production system.

3. Application / Course Management Module

Assessment: The module demonstrates the use of Python backend logic and database operations for creating, viewing, updating, and deleting application or course records. REST API endpoints can be used to connect the backend with the frontend.

Areas for Improvement: File uploads, advanced search and filtering, pagination, and richer content management can be added to improve usability and scalability.

Module Assessment Summary

Module	Assessment Area	Overall Level	Recommended Improvement
Authentication	Registration, login, logout, protected routes	Good	Add password recovery and stronger account security
User Management	User/profile data and CRUD operations	Good	Add role management and account controls
Frontend	HTML, CSS, Bootstrap, JavaScript and forms	Good	Add richer interactive components and accessibility improvements
Backend	Python routes, validation, business logic and APIs	Good	Add advanced error handling, logging and API documentation
Database	Persistent storage and CRUD operations	Good	Add advanced indexing, backup and scalability strategies
Testing	Functional, performance and negative test cases	Good	Automate tests and increase load-test coverage

Overall Assessment: The Python Full Stack internship provided practical exposure to the complete web application development lifecycle, including frontend design, Python backend development, database integration, API communication, testing, debugging, and basic security practices.

Conclusion

The Python Full Stack Developer Virtual Internship provided a valuable practical learning experience in developing complete web applications by integrating frontend, backend, and database technologies. The internship helped in understanding the complete application development process, from requirement analysis and interface design to backend implementation, database integration, testing, debugging, and final documentation.

Python was studied as the core programming language for backend development. The internship also provided an understanding of how frontend technologies such as HTML5, CSS3, JavaScript, and Bootstrap can be integrated with Python-based server-side logic to create responsive and interactive web applications.

The internship strengthened the understanding of database concepts and CRUD operations. The application architecture demonstrates how user requests are received by the frontend, processed by the Python backend, and connected with a database for storing and retrieving information. REST API concepts and JSON-based communication were also considered for connecting different layers of the application.

Testing and debugging formed an important part of the development process. Functional testing, negative testing, API testing, and performance considerations helped in identifying possible errors and improving application reliability. The use of version control concepts also provided an understanding of maintaining and managing source code during software development.

Although the internship was limited to the scope of a virtual learning and development program, it provided a strong foundation for further exploration of Python web frameworks, database systems, REST APIs, cloud deployment, automated testing, and secure software development.

Overall, the Python Full Stack Developer Virtual Internship was a significant learning experience that improved practical knowledge of full stack web development and helped in understanding how different technologies work together to build functional software applications. The knowledge and skills gained during the internship will be useful for future academic projects and professional software development activities.

Bibliography

- Python Software Foundation. Python Documentation. <https://docs.python.org/3/>
- Pallets Projects. Flask Documentation. <https://flask.palletsprojects.com/>
- Django Software Foundation. Django Documentation.
<https://docs.djangoproject.com/>
- Oracle Corporation. MySQL Documentation. <https://dev.mysql.com/doc/>
- SQLite Consortium. SQLite Documentation. <https://www.sqlite.org/docs.html>
- Mozilla Foundation. MDN Web Docs. <https://developer.mozilla.org/>
- Bootstrap Team. Bootstrap Documentation. <https://getbootstrap.com/docs/>
- Git Community. Git Documentation. <https://git-scm.com/doc>
- GitHub. GitHub Documentation. <https://docs.github.com/>
- Postman. Postman Documentation. <https://learning.postman.com/docs/>
- SQLAlchemy Project. SQLAlchemy Documentation. <https://docs.sqlalchemy.org/>
- Mozilla Foundation. HTTP Documentation – MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP>
- Scott Chacon and Ben Straub. Pro Git Book. <https://git-scm.com/book/en/v2>